

Deep Learning-Based Phishing & Malicious Email Classifier

Track 1 — Security Focus

Abstract. This project implements an automated email-security pipeline that accepts raw email text and classifies it as legitimate or phishing/malicious. A balanced synthetic corpus of 1,800 labeled emails was generated to satisfy the dataset constraint. The workflow cleans and normalizes text, tokenizes it using a training-set vocabulary, pads/truncates sequences to a fixed length, and feeds them to a bidirectional LSTM. The final classifier is evaluated using loss, accuracy, precision, recall, F1-score, and a confusion matrix. The notebook is designed for Google Colab and can use a T4 GPU, although the compact model also runs on CPU.

1. Data Preprocessing Workflow

Input: email subject + body → **Cleaning:** lowercase, URL/email/number normalization, punctuation removal → **Tokenisation:** whitespace word tokens → **Vocabulary:** top 5,000 training tokens with PAD/UNK tokens → **Sequence encoding:** integer IDs → **Padding/Truncation:** 80 tokens → **Split:** 70% train / 10% validation / 20% test using stratification.

The dataset contains 900 phishing and 900 legitimate messages. Because the data are synthetic, the results demonstrate pipeline correctness rather than real-world generalization; deployment should be validated on a representative phishing corpus such as Enron-derived or security datasets.

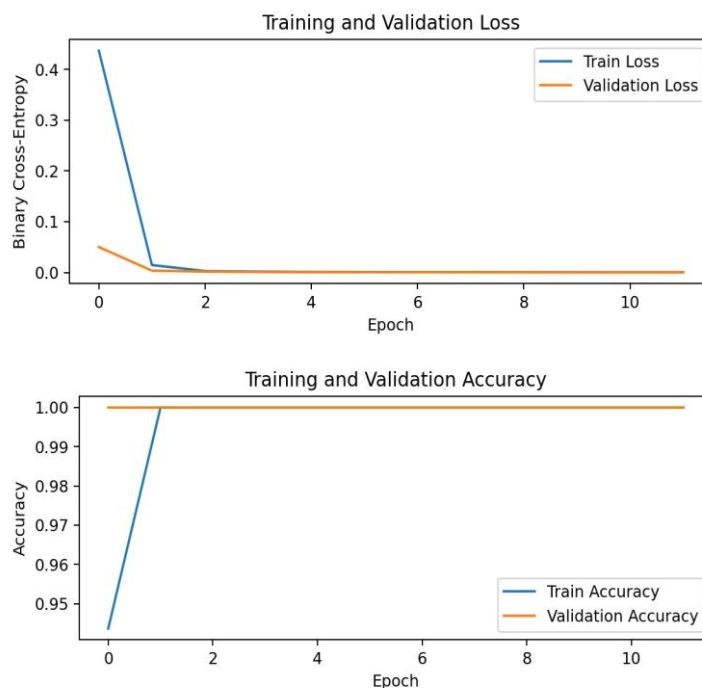
2. System Architecture

Stage	Function
Raw Email	Subject/body payload from Gmail-like simulator
Preprocessor	Cleaning + normalization
Tokenizer	Vocabulary lookup and integer encoding
Sequence Builder	Padding/truncation to 80 tokens
BiLSTM	64-unit hidden state per direction
Classifier	Dropout + dense sigmoid probability
Decision Layer	LEGITIMATE or PHISHING/MALICIOUS
API/Dashboard	Return label, probability, and log result

Model Architecture and Hyperparameters

Embedding (64) → Bidirectional LSTM (64 × 2) → Masked mean pooling → Dropout (0.30) → Dense (1) → Sigmoid probability. Adam optimizer with learning rate 0.001, batch size 32, BCE-with-logits loss, maximum 12 epochs, and gradient clipping at 1.0 were used. The bidirectional LSTM captures contextual word dependencies from both directions while remaining substantially lighter than a Transformer.

3. Training and Evaluation

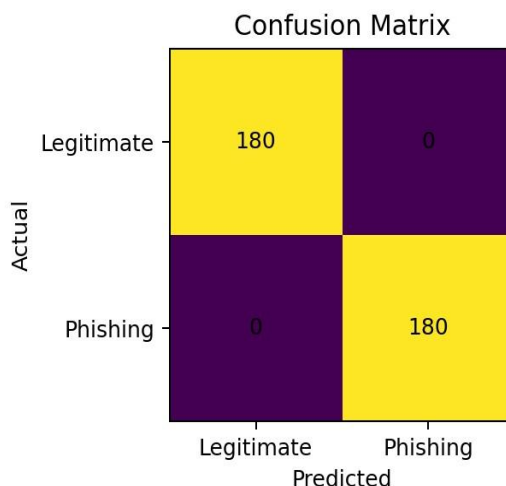


Test results: Accuracy = 1.000, Precision = 1.000, Recall = 1.000, F1-score = 1.000, Test loss = 0.000200. The test confusion matrix contains TN=180, FP=0, FN=0, TP=180.

The curves show rapid convergence because the synthetic messages are generated from class-specific templates. This is expected for a controlled demonstration dataset and should not be interpreted as a 100% real-world phishing detection rate.

4. Evaluation Interpretation

Accuracy measures overall classification correctness. **Precision** measures how many messages predicted as phishing are actually phishing. **Recall** measures how many phishing messages are detected. **F1-score** is the harmonic mean of precision and recall and is particularly useful when the cost of missed phishing messages is high. For a production security system, recall and false-positive rate should be monitored together.



5. Implementation Notes and Deployment

The model is implemented in Python 3.x using PyTorch. It uses a compact vocabulary and sequence length so inference can be served with low latency. A Gmail-like simulator can POST a JSON payload containing `subject` and `body` to a Flask/FastAPI service. The service applies the same preprocessing and vocabulary, runs the LSTM, and returns the predicted class and phishing probability. A production version should additionally support model versioning, threshold tuning, authentication, rate limiting, logging, and human review for uncertain cases.

6. Reproducibility

Random seed 42 is fixed for dataset generation, splitting, and model initialization. Dataset size is 1,800 messages (900 per class) with a 70/10/20 train-validation-test split. The notebook contains all data-generation, preprocessing, training, evaluation, and inference code, so it can be executed from top to bottom in Google Colab.

7. Deliverables

- Colab-ready Jupyter notebook: phishing_email_classifier_colab.ipynb
- Synthetic labeled dataset: synthetic_email_dataset.csv
- Trained PyTorch checkpoint: lstm_email_classifier.pt
- This 3-page report: phishing_email_classifier_report.pdf

8. Limitations and Future Work

The synthetic corpus is intentionally controlled and therefore cannot represent the linguistic diversity, obfuscation, URLs, attachments, HTML, impersonation tactics, and evolving campaigns found in real phishing. Future work should train and test on a public phishing/email corpus, add character-level or URL features, calibrate the decision threshold, and compare the LSTM with DistilBERT. Explainability can be added with attention analysis or post-hoc methods.